CEG 4330/6330 – Microprocessor-based Embedded Systems

Project 5 – ESP32 and Networking

Learning Objectives
The purpose of this project is for you to learn how to design a system that allows a Python program and an embedded system to communicate over a network.

Setup
To use the Python client, you will have to install Python here: https://www.python.org/downloads/

Next, via command line you'll have to install OpenCV to display the image received by the ESP32:

pip install opencv-python

WiFi Camera Server
The camera sample code provided includes the code necessary to transmit the camera feed from the ESP32-WROVER-E via WiFi to the Python client. You can adjust the frame size and jpeg quality as listed in the configuration. Setting the frame size and quality lower will result in faster transmission and increasing them will improve quality at the cost of transmission speed.

JSON Format
JSON is a way of formatting data in a way that is easy to read for humans but also has a defined structure using brackets to make it consistently readable by computers. An example of JSON format for a student's data:

```
{
  "name": "John Doe",
  "age": 29,
  "isStudent": false,
  "email": null,
  "skills": ["Embedded Systems", "C", "Python"],
  "address": {
    "city": "Dayton",
    "state": "OH"
  },
  "projects": [
    { "title": "Barcode Reader", "completed": true },
    { "title": "Remote Controlled Car", "completed": false }
  ]
}
```

Rework your code from Project 2 to add a debounced button to this project's provided code and store both the number of button presses and the button press time inside of a JsonDocument object. Then, print the JSON object to the serial monitor if a button press occurs. ***Demonstrate the printed serial output of the JSON to the TA.

Sending JSON to Python Client

Start the provided Python program on a computer that is on the same network as your ESP32. Next, you must transmit the JSON data stored on the ESP32 to the Python client via a TCP socket. You should send this JSON data every time the button is pressed. If you serialize and send your JSON object properly, it will be displayed in the terminal that is running the Python program. To implement this program, complete the following steps:

1. Write the “connectToWifi” function to begin a WiFi connection on a specified SSID and password:
    a. The function is required to attempt to connect to a predefined SSID and password and if after some time the WiFi is still not connected, print an error message and retry the connection.
    b. You are not allowed to use delay() to wait to see if the WiFi can be connected to, you may use millis().
    c. For using the WiFi functionality of the ESP32 make sure you use the library “#include <WiFi.h>”.
    d. The following functions will be helpful:
        i. WiFi.status() - returns “WL_CONNECTED” if properly connected to WiFi
        ii. WiFi.begin(ssid, password) - attempts to connect to the provided SSID with password
        iii. WiFi.disconnect() - disconnects from the WiFi (call before you retry after failing to connect)
2. Write the “startJSONServer” function to begin a TCP server on the WiFi network
    a. If the WiFi status is not connected, return.
    b. Otherwise call the following functions to start the server:
        i. server = WiFiServer(port);
        ii. server.begin();
    c. Write a simple test in the loop() function to show that the server can send data and receive data from a single client.
        i. To create a TCP client that can connect to your ESP32 you can use the provided Python program
        ii. Use WiFiClient client = server.available(); to see if a client is connected
        iii. Use client.available(); to see if the client sent data
        iv. Use receivedData = client.readStringUntil('\n'); to read the data the client sent
        v. Use the client.println(“test”); to send a string to the client
3. Connect an external push button to an input pin on the ESP32 with an internal pull up and ground.
4. Write the “loop” function
    a. If a WiFiClient is connected and the button state has been pressed (with debouncing), track the button presses, then update the internal JSON object.
        i. Use digitalRead() to check a button’s state and increment the total amount of times the button was pressed
        ii. Use json["buttonPresses"] = buttonPresses; to update the button state in the JSON
        iii. Use json["buttonPressTime"] = buttonPressTime; to update the button presses in the JSON
        iv. Use serializeJson(json, jsonString); to “serialize” or convert the JSON object to a string
        v. Use client.println(jsonString); to send the serialized JSON string to the Python client

***Demonstrate JSON output to the TA.

Receiving JSON from Python Client

The Python program is already written to send a JSON formatted string which contains a frequency and duty cycle to turn on the LED using PWM. On the ESP32, read the incoming JSON string over TCP, deserialize it, and control the onboard LED using PWM with the frequency and duty cycle that is received.

1. Write the “void handleJSONFromClient(WiFiClient client)” function to receive data from the Python server:
    a. Read the data from the client:
        i. String receivedData = client.readStringUntil('\n').trim();
    b. Deserialize the string into a JSON object and return early if an error has occurred:
        i. DeserializationError error = deserializeJson(rx, receivedData);
    c. Get the frequency and duty cycle from the JSON object:
        i. if (rx["frequency"].is<float>() && rx["dutyCycle"].is<float>())
        ii. float freq = (float)rx["frequency"];
    d. Set the LED PWM frequency and duty cycle based on the

***Demonstrate the JSON output to the TA.

Replying to Python Client

CEG6330 Students Only: Once you have received the JSON data from the Python client, reply to the Python client with the actual frequency and duty cycle achieved. Due to limitations in the timer’s resolution and frequency that you decide, a perfect frequency and duty cycle will not be perfectly attainable. ***Demonstrate the JSON being received on the Python program to the TA.

How to Submit

Demonstrate the various milestones mentioned above to the TA in person. Submit all programs to Pilot.

Grading

This lab is worth 5.00 points, distributed as follows:

Task	Points
ESP32 transmits JSON to serial monitor	1.00
ESP32 transmits JSON to Python client	2.00
ESP32 receives JSON from Python client and updates LED PWM	2.00
CEG 6330: ESP32 responds to Python client	20% total
Total	5.00